LAB.2

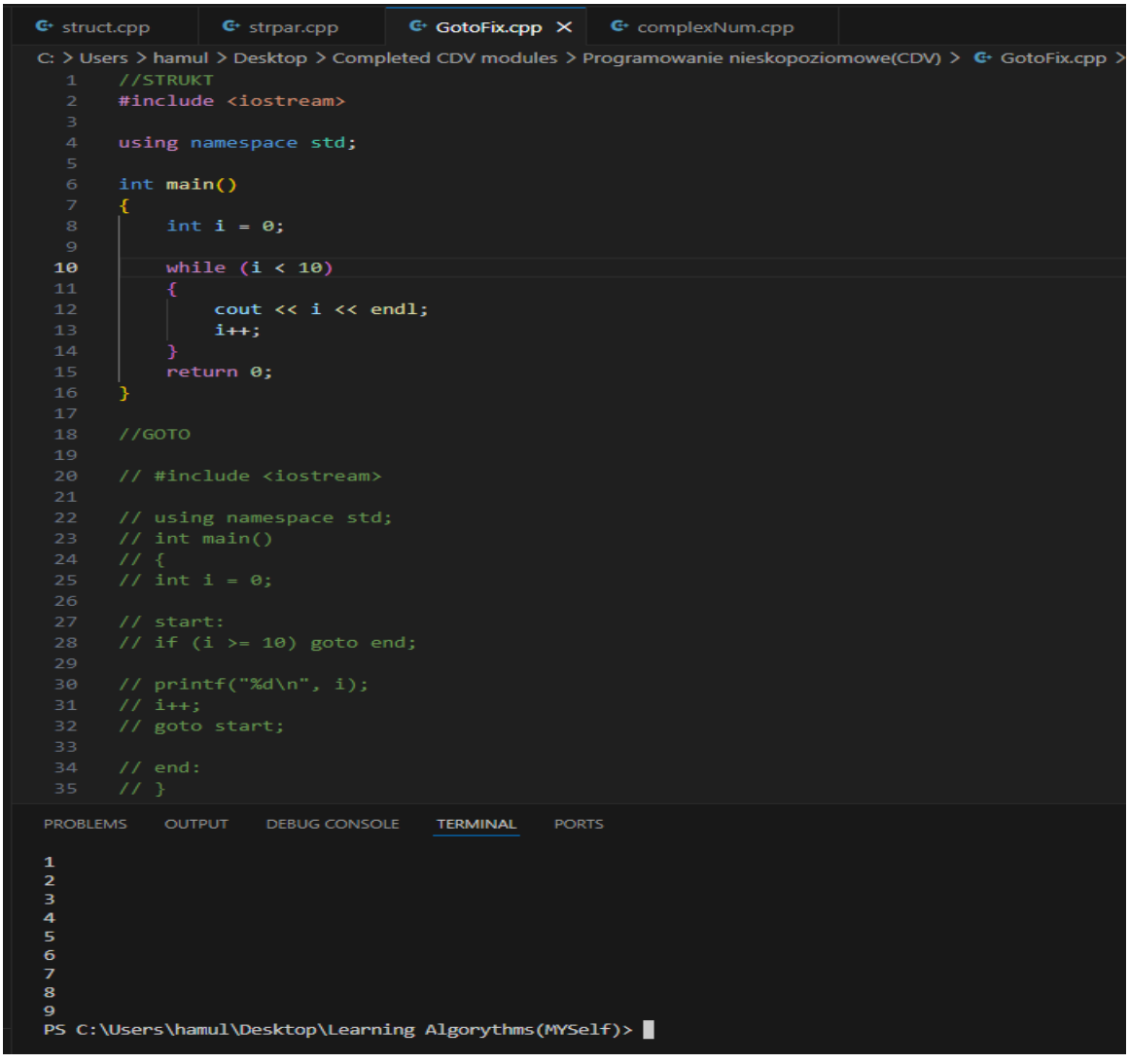
1.

Instrukcja **goto** nie jest zalecana przez to że ma słabą czytelność kodu, trudno w niej szukać błędy. Użycie **goto** to jak czytanie książki w której co drugą stronę masz napisane: „a teraz skocz do strony 58”, a na stronie 58: „zignoruj to i wróć do strony 12”.
Przez takie niekontrolowane "skakanie" po programie bardzo trudno jest śledzić jego logikę, szukać błędów(debugować) oraz rozwijać i utrzymywać projekt. Zamiast tego łatwo o przypadkowe błędy logiczne.

2.

Spaghetti code to potoczne określenie kodu źródłowego, który jest chaotyczny, nieustrukturyzowany i przez to niezwykle trudny do zrozumienia oraz modyfikacji. Nazwa bierze się stąd, że ścieżki wykonania programu są poplątane niczym nitki makaronu na talerzu.
Przykład sytuacji:
Taki kod powstaje najczęściej, gdy programista nadużywa instrukcji skoku **goto,** unika dzielenia programu na mniejsze, logiczne funkcje, na przykład pisząc kod w jednej dużej funkcji **main**. Oraz tworzy bardzo głębokie zagnieżdżenia, pętle w pętlach i instrukcje **if** wewnątrz innych **ifów.** W efekcie zmiana jednej linijki często psuje działanie innej, pozornie niezwiązanej części programu.

3. Wykorzystałem pętle while, dzięki temu kod stał się krótszy, o wiele bardziej czytelny i zgodny z dobrymi praktykami programowania.



4.

Używałem pętli for która przechodzi przez całą tablicę, jeśli element zostanie znaleziony **tab[i] ==x** ustawiam flagę **found = 1** i używam instrukcji **break** aby natychmiast przerwać działanie pętli. Kod jest teraz logiczny i bezpieczny.



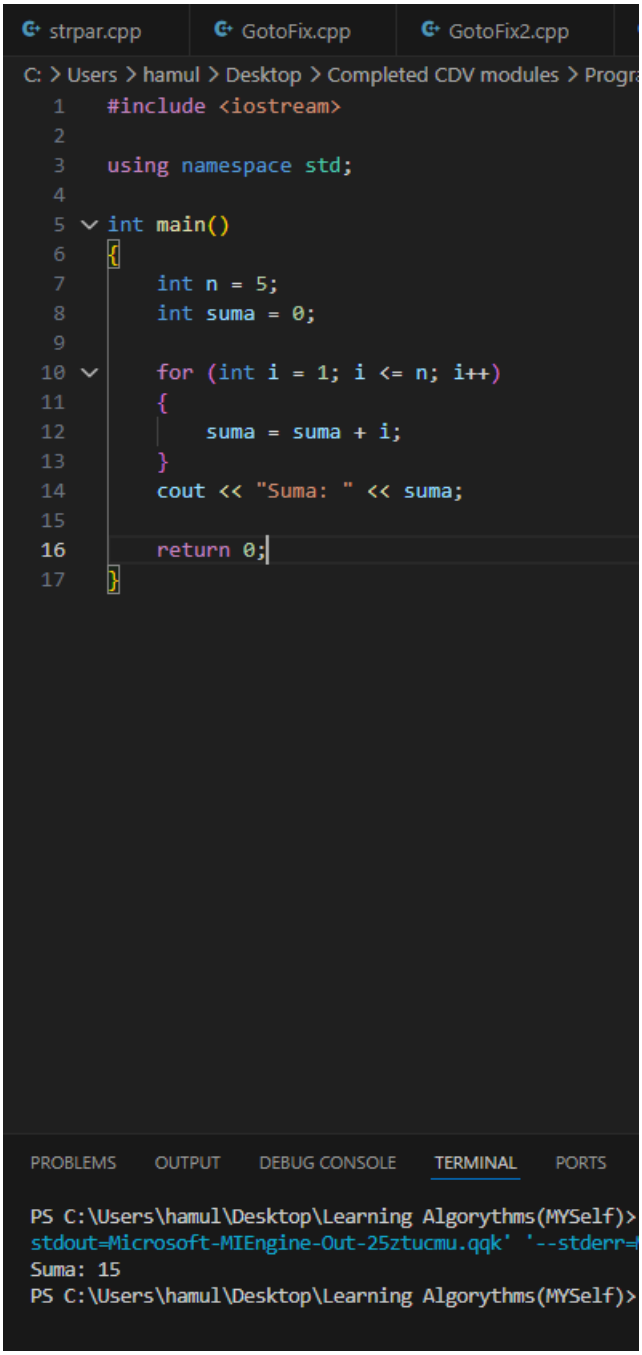
5.

Porównanie rozwiązań(goto vs pętla):

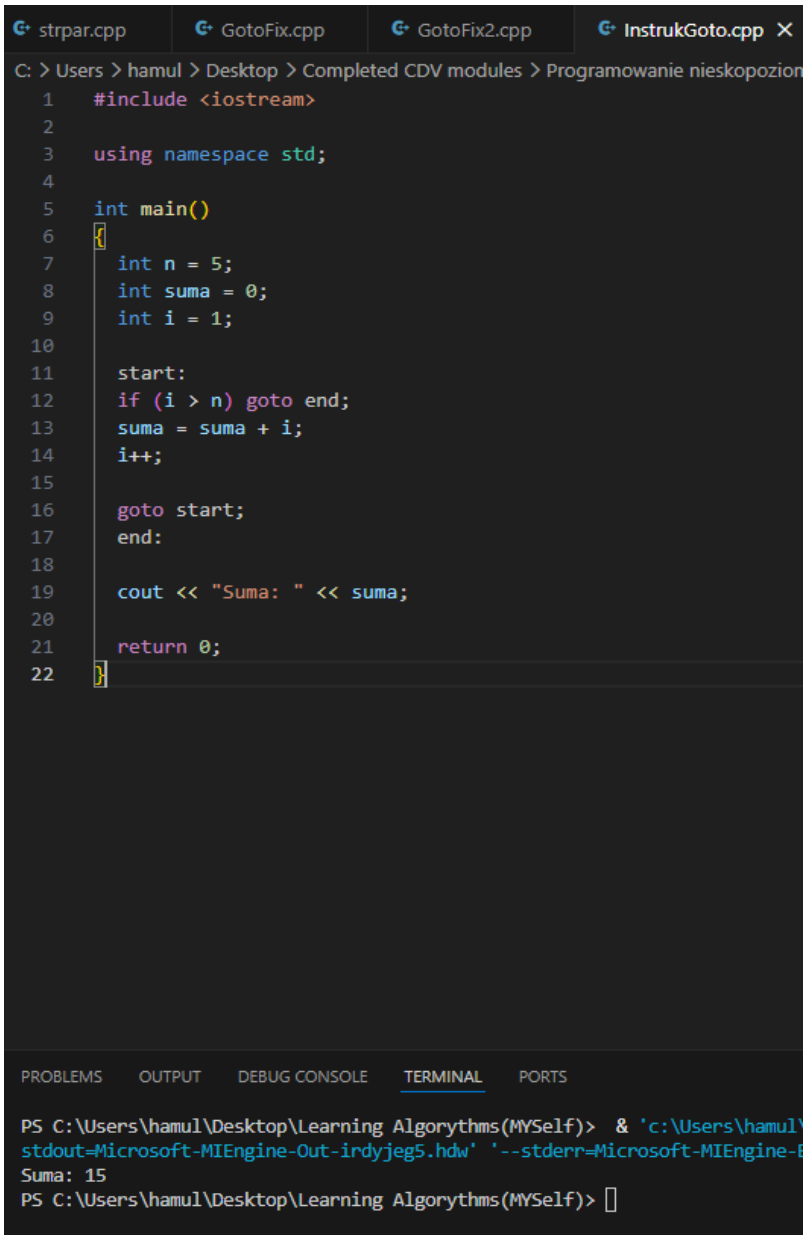
Czytelność: Wersja z pętlą **for** jest zdecydowanie bardziej czytelna. Od razu widać w niej warunki brzegowe, wszystko jest w jednej linijce. Kod z użyciem **goto** wymusza skakanie wzrokiem po linijkach w poszukiwaniu etykiet co przy większym programie prowadzi do nieczytelnego "spaghetti code".

Łatwość modyfikacji:Pętla jest znacznie bezpieczniejsza i łatwiejsza do zmiany. Jeśli chcemy zmodyfikować kod po prostu dopisujemy ją wewnątrz bloku **{}.** W przypadku **goto,** dodawanie nowego kodu zwiększa ryzyko zepsucia logiki skoków, co może łatwo doprowadzić do powstania nieskończonej pętli lub pominięcia ważnych instrukcji. Z tego powodu w nowoczesnym programowaniu unika się **goto.**

Pętla



GOTO



6.

Zgodnie z zasadami programowania strukturalnego, zrefaktoryzowałem podany kod, dzieląc go na mniejsze, niezależne funkcje. Główny blok programu **main** został odciążony i służy teraz jedynie do wczytywania danych oraz sterowania przepływem. Utworzyłem dwie funkcje obliczeniowe **dodaj** i **odejmij** które realizują pojedyncze operacje matematyczne i zwracają wynik, oraz oddzielną funkcję **wypisz(void)** która odpowiada wyłącznie za wyświetlanie danych na ekranie.

